

Representing Software Usage Models with Stochastic Automata Networks

André G. Farina
CPTS-PPGCC-PUCRS
FACIN – Av. Ipiranga, 6681
Porto Alegre BRAZIL
farina@cpts.pucrs.br

Paulo Fernandes
PPGCC-PUCRS
FACIN – Av. Ipiranga, 6681
Porto Alegre BRAZIL
paulof@inf.pucrs.br

Flávio M. Oliveira
CPTS-PUCRS
FACIN – Av. Ipiranga, 6681
Porto Alegre BRAZIL
flavio@cpts.pucrs.br

ABSTRACT

The benefits of usage models in the statistical testing of software have been recognized by the software engineering community and discrete state formalisms have been largely used to describe such usage models of software. The Markov Chains formalism (MC) is a natural choice to do so. However, in this paper we suggest the use of a more sophisticated discrete state formalism, the Stochastic Automata Networks (SAN). In many problems of the performance evaluation area, the SAN models present advantages over MC models. Therefore, it seems natural to us to verify similar advantages in the modeling of usage models.

This paper presents a case study of building a software usage model using SAN formalism. A software tool called DOCSEDITOR is modeled using MC and SAN. The models are compared in terms of number of states, scalability, and readability. It is not the objective of this paper to present a full framework to develop and analyse usage models with SAN, but just to show some evident advantages of the use of SAN instead of MC. In order to do this, the conclusion points out the indexes than can be computed from both models and suggests the next steps on future work.

Categories and Subject Descriptors

D.2 [Software]: Software Engineering; D.2.4 [Software Engineering]: Software/Program Verification—*Statistical Methods*; G.3 [Mathematics of Computing]: Probability and Statistics—*Markov Processes*

General Terms

Measurement, Reliability, Verification

Keywords

Statistical Testing, Usage Models, Stochastic Automata Networks

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy, otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

SEKE '02, July 15-19, 2002, Ischia, Italy.

Copyright 2002 ACM 1-58113-556-4/02/0700...\$5.00.

1. INTRODUCTION

The complexity of software development demands the use of sophisticated formal methods to describe it. This concern with a formal description of a software is certainly useful in all phases of a software life-cycle. In this paper, we are particularly interested in formal tools to support the software testing procedures [6]. However, the technique discussed here may be extended to the other phases.

The software engineering community has recently turn its attention to *usage models* in statistical software testing [10]. It seems natural to consider the description of a software as a discrete state model [5]; therefore, Markov chains are the simplest way to describe it. Markov chains is one of the oldest formalisms used to describe discrete state models [11, 14]. A Markov chain model describes a system as a set of possible states and transitions among them. Each transition is described by a stochastic process with exponential distribution¹.

For almost a decade the software engineering community has been using Markov chains to describe usage models [12, 13, 15, 16]. In this paper we suggest the use of a more sophisticated discrete state formalism, the Stochastic Automata Networks (SAN). SAN is a formalism with the same power of description as the Markov chains; however, in SAN a system is not described as a single system with states and transitions in a single level. A system is described by SAN formalism as a collection of subsystems described by local states and transitions among the states. Each of the subsystem transitions can be affected by the states or transitions of the other subsystems, therefore providing interactions among the subsystems. The composition of all the subsystems is equivalent to a, usually huge, Markov chain model.

In many problems of the performance evaluation area, the SAN models present advantages over Markov chains models. The purpose of this paper is to verify whether similar advantages can be found in the modeling of usage models. In order to do so, we present a case study of modeling a

¹Markov chains can be used with a discrete or continuous time scale. For the purposes of this paper is not important if discrete-time or continuous-time scale Markov chains are used. The reader interested in the differences and similarities between this two kinds of Markov chains can found a large material in [11].

software usage model using SAN formalism. A software tool called DOCSEDITOR is modeled using Markov chains and SAN. The models are compared in terms of number of states, scalability, and readability. It is not the objective of this paper to present a full framework to develop and analyse usage models with SAN, but just to show some evident advantages of the use of SAN instead of Markov chains.

The next section (Section 2) presents some basic concepts on usage models. Section 3 presents an informal definition of SAN formalism. Section 4 draws some modeling considerations, presents the DOCSEDITOR software and its Markov chains and SAN models. Since this paper presents a case study, the conclusion points out the indexes that can be computed from both models and suggests the next steps on future work.

2. USAGE MODELS

A usage model characterizes the operational use of a software system. Three aspects can be considered in the construction of a software usage model: *the software use, the user, and the environment of use*. It can be considered *software use* a working session, a transaction or any service unit limited by a start/finish event that defines an usage instance. The *user* of a software can be a person, a hardware device or another software without any loss of generality. The *environment of use* can be defined by the platform (OS, window manager, etc), number of users (single/multiple-user), concurrent applications (distributed, parallel, etc) and other aspects that may affect the software behavior.

The model structure is composed of a state set and the transitions between these states, constituting a graph. The graph nodes represent the model states, and the graph arcs represent the transitions between states. This structure describes the possible uses of the software. A probability distribution, associated with the model, describes the expected use of the software.

Therefore, usage models can be represented by discrete-state system modeling formalisms [10] and Markov chains are the natural choice. In this formalism, the usage states of the software are mapped into states and the user actions represent transitions among those states. The transition probabilities² represent the expectation on user actions and therefore they configure the usage pattern of the modeled software. Due to the characteristics of usage models the Markov chain must have two special states representing the beginning and the end of the software use. These are the only start and finish states of the Markov chain.

The usage model may be applied at many stages of the software life-cycle, in order to refine system specification, evaluate complexity, drive the verification efforts, identify some

²Without any loss of generality discrete-time or continuous-time models can be used to model software usage. If discrete-time scale is adopted the transitions will have probabilities of occurrence in discrete time tics. If continuous-time scale is adopted transitions will have rates describing the frequency in which they occurs. The reader interested in the differences and similarities of discrete-time and continuous-time Markov chains can consult [11] for further information.

event's frequency, estimate the testing effort, estimate software reliability, and so on. We focus here on its application to the testing phase and reliability estimation.

Represented with Markov chains, usage models allow the test engineer, *i.e.*, a person who has the responsibility of test creation and management, to predict the critical paths, more susceptible to failure, concentrating the efforts in this context. This analysis is performed over the occurrence probabilities associated to each use of the software.

From the usage model analysis it is possible to extract several interesting properties, such as [13]:

- Number of software statistically typical usage paths;
- Long-run occupancy, e.g. utilization time percentage for each state;
- Mean number of events per test case;
- Mean number of events between two states.

3. STOCHASTIC AUTOMATA NETWORKS FORMALISM

In this section we introduce the basic concepts of the SAN formalism without a formal description. The reader interested in a formal description of the formalism can consult the previous publications about it [4, 9], or the original works on the subject [3, 8].

The SAN formalism describes a complete system as a collection of subsystems that interact with each other. Each subsystem is described as a stochastic automaton, *i.e.*, an automaton in which the transitions are modeled by a continuous-time stochastic process³. The state of a SAN model, called *global state*, is defined by the combination of the state of each automaton, called *local states*. Figure 1 represents a SAN model with 2 automata completely independent and its equivalent Markov chain.

Note that in the model of Figure 1 there is no interaction between the two automata. In the following sections we will extend this model to illustrate the use of the two SAN formalism primitives to represent the interactions among automata: the *synchronized events* and the *functional rates*.

3.1 Synchronized Events

Two types of events can change the global state of a SAN model: *local events* and *synchronized events*. The local events change the global state by changing only one local state, *i.e.*, passing from a global state to another that differs only by one local state. The synchronized events can change simultaneously more than one local state, *i.e.*, two or more automata can change their local states simultaneously. The model in Figure 1 has only local events, Figure 2 represents a slightly different model where a synchronized event has been included.

³Queueing Networks formalism usually describe the interarrival and service durations in a continuous-time scale. Therefore, in the context of this paper only continuous-time SAN will be considered, but discrete-time SAN can also be employed without any loss of generality.

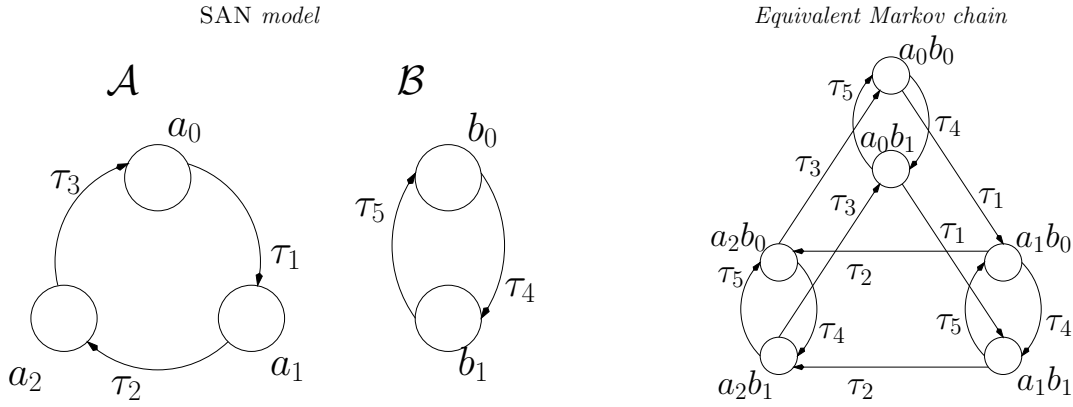


Figure 1: A SAN model with 2 independent automata

The occurrence of a synchronized event *forces* all automata concerned to fire a transition corresponding to this event. Thus, the transitions representing a synchronized event will no longer be represented by a single rate (a non-negative real number), but those transitions will be represented by the *name* of the synchronized event (an identifier), its *firing rate* and its *probability of occurrence*:

- The *name* of the event is necessary to identify which transitions must fire simultaneously.
- The *firing rate* describes the rate in which the event occurs, but since it appears in all automata concerned by the event, one single automaton is chosen to indicate the event rate⁴. All other automata concerned by the event will indicate a symbolic rate equal to 1.
- The *probability of occurrence* establishes a relationship among all transitions corresponding to a same event that can be fired from the same local state, and therefore cannot be fired simultaneously.

In the model of Figure 2 the transitions: $a_0 \rightarrow a_1$, $a_1 \rightarrow a_2$, $a_2 \rightarrow a_0$, and $b_0 \rightarrow b_1$ represent local events. The transitions: $a_2 \rightarrow a_0$, $a_2 \rightarrow a_1$, and $b_1 \rightarrow b_0$ represent the synchronized event s . Note that the transition $a_2 \rightarrow a_0$ can be fired by the occurrence of a local event as well as by the occurrence of the event called s . In this example, the occurrence of the synchronized event s (which will happen at rate τ_5) will cause one of two situations:

- automata $\mathcal{A}$ will pass from state a_2 to state a_1 and at the same time automata $\mathcal{B}$ will pass from state b_0 to state b_1 (with probability π_1); or
- automata $\mathcal{A}$ will pass from state a_2 to state a_0 and at the same time automata $\mathcal{B}$ will pass from state b_0 to state b_1 (with probability $1 - \pi_1$).

⁴This particular choice to indicate a rate of a synchronized event only in one automaton is arbitrary and it does not affect the semantics of the SAN model. More detailed information about synchronized events can be found in [3].

3.2 Functional Rates

The use of functional rates is the second form of interaction among automata. A functional rate is no longer a non-negative real number (like the ordinary rates), but a discrete function of the local state of some automata over the non-negative real numbers. Figure 3 represents a variation of the example in Figure 2. In this new example, the transition of the state b_0 to the state b_1 is no longer independent of the automaton $\mathcal{A}$, but now it is a function called f defined as:

$$f = \begin{cases} \lambda_1 & \text{if automaton } \mathcal{A} \text{ is in the state } a_0; \\ 0 & \text{if automaton } \mathcal{A} \text{ is in the state } a_1; \\ \lambda_2 & \text{if automaton } \mathcal{A} \text{ is in the state } a_2; \end{cases}$$

In this model, the firing of the transition from state b_0 to b_1 will occur with rate λ_1 if automaton $\mathcal{A}$ is at state a_0 , or with rate λ_2 if automaton $\mathcal{A}$ is at state a_2 . If automaton $\mathcal{A}$ is at state a_1 the transition from state b_0 to b_1 will not occur. According to the SAN formalism, the expression of this function can be:

$$f = [\lambda_1 (st(\mathcal{A}) = a_0)] + [\lambda_2 (st(\mathcal{A}) = a_2)]$$

The interpretation of a function can be viewed as the evaluation of an expression of non-typed programming languages, *e.g.*, C language. Each comparison is evaluated to 1 for true values and to 0 for false values.

Note that the use of functional rates is not limited to local events. In fact, for synchronized events not only the event rate, but also the probability of occurrence, can be expressed as a function. The use of functional transitions is a powerful primitive of the SAN formalism, since it allows us to describe very complex structures in a very compact format. The computational costs to handle functional rates has decreased enormously with the recent developments of numerical solutions for SAN models, *e.g.*, the algorithms for generalized tensor products [4].

3.3 Performance Indexes

A SAN model can be used to compute stationary measures based on the computation of the eigenvector of the equivalent Markov chain. Therefore, integration functions over the resulting probability vector can be applied. Reward values can be associated to the probability of every and each global state. The format of such integration functions in SAN formalism is similar to the definition of functional rates. In the

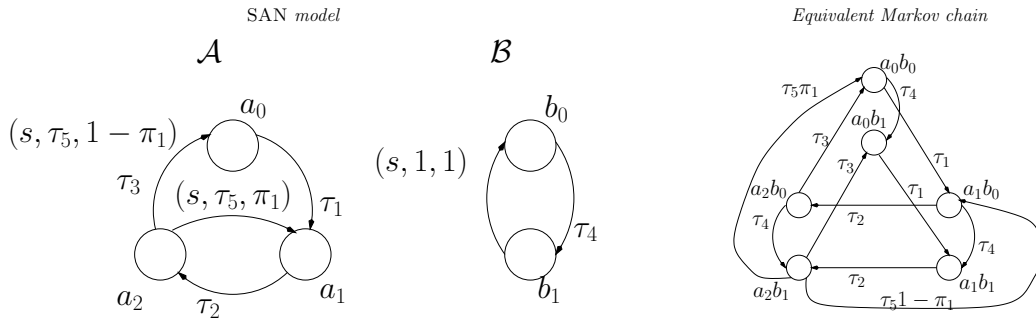


Figure 2: A SAN model with 2 automata and 1 synchronized event

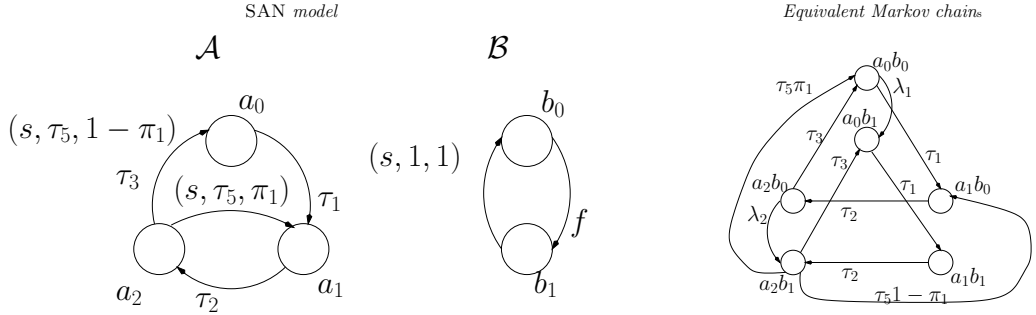


Figure 3: A SAN model with 2 automata, 1 synchronized event, and 1 functional rate

examples expressed in Figures 1, 2, and 3, integration functions can be defined to compute the probability of each local state. For example, the integration function “ $st(\mathcal{A}) = a_0$ ” could be used to compute the probability of the local state a_0 , *i.e.*, the sum of the probabilities of all global states where the automaton $\mathcal{A}$ is in the state a_0 .

4. DOCSEDITOR CASE STUDY

Stochastic automata networks have shown a number of advantages over Markov Chains for modeling complex systems [4, 11]. We believe that this is the case for statistical testing based on usage models. In fact, the use of SAN instead of straight forward Markov chains represents no loss of generality since both formalisms have the same power of modeling [3]. The knowledge of SAN formalism allow us to believe that usage models described with SAN have some interesting characteristics:

- environment requirements (a critical issue for testing) can be made explicit in the model by the assignment of an automaton to each requirement;
- the representation is modular, improving maintainability and readability;
- an individual use is a sequence of global states in the SAN - thus, its description is more detailed, which allows an easier mapping of usage pattern into test cases.

In order to investigate those beliefs, we performed a preliminary case study, which consisted of the following steps:

1. the choice of an example application;

2. the description of a Markov model structure (state and transitions);
3. the gathering of usage statistics;
4. the assignment of probabilities to the Markov model;
5. the description of an equivalent SAN model;
6. the comparison of both models.

The application we selected was DOCSEDITOR, a generic forms-based document editor developed at CPTS project [1] where the authors participate. This editor is currently used to create and maintain test-related information, *e.g.*, test plans, bug reports. We found that DOCSEDITOR has the adequate level of complexity for this first case study. Also, since it was developed in our project, it was easy to instrument the source code to generate the necessary usage logs. The next section describes the DOCSEDITOR software.

4.1 DOCSEDITOR Software

The Generic Forms Based Documents Editor, called DOCSEDITOR, is a tool that allows the edition of structured, object-oriented, form-based documents. In DOCSEDITOR, a document is composed by a set of object forms. Each form guides the user to keep the text and the information in a pre-defined style and format. Figure 4 illustrates the application’s interface and presents the system architecture.

A document, for DOCSEDITOR, is an instance of some document type. A document type describes the different kinds of forms that a document, of some class, should contain and the hierarchical relationships between these forms (a tree). The

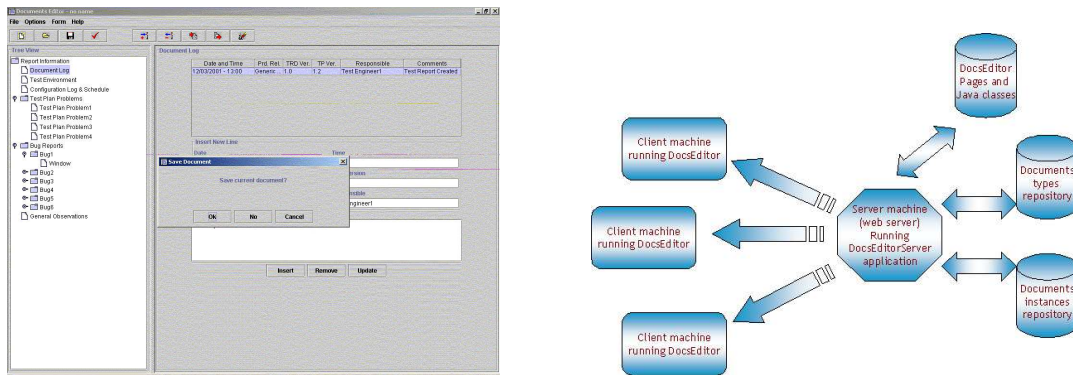


Figure 4: DOCSEDITOR Interface and System Architecture

structure of a document is always hierarchical, allowing subsections nested into sections in as many levels as needed. A document instance is composed of an instance of the structure tree (which includes a list of all used forms) and the contents of the forms. When the document is loaded, the editor is capable of loading all the forms used to edit the document.

The DOCSEDITOR is a client server application composed by two types of modules: the server module and the editor module. The server module is responsible to save, restore and control the ownership of documents edited by instances of the editor module. The editor module allows the user to choose the document type he/she wants to work with, to edit a document and to send or retrieve documents from a running instance of the server module.

The system administrator must register the users of the editor organized in groups with document sharing permissions Unix-like defined.

4.2 The Experiment

The goal of the experiment described here was to evaluate the characteristics and potential benefits of Markov chains and SAN formalism in building usage models for statistical software testing. We focus here in the characteristics relative to the model construction process, a complete description of the experiment and both the Markov chains and the SAN models can be found in [2].

We developed two model instances of the same DOCSEDITOR usage model, one with Markov chains and one with SAN. The model construction process and the model information are the source for the comparison between the formalisms. It was necessary to collect information about the application's usage, therefore, an event log to register the all relevant user actions was implemented. The registry is local, generating a log file for each user of the application. In terms of model delimitation, the usage model described in this experiment do not consider the object tree of the application, nor the instanced forms.

The elements considered as model states take into account the following characteristics of the software:

- the dialog windows that may appear during DOCSEDITOR execution (eight different kinds of dialogs are possible);
- the selection window, *i.e.*, the hierarchical (tree) representation of forms;
- the document state (no active document, titled document active and untitled document active);
- the server availability; and
- the browser availability.

The application's commands are considered as transitions among such states.

4.2.1 Model 1 – Markov Chain

The model structure, using Markov chain formalism, is based on a state machine description of the application according to the system specification. The proposed model describes the evolution of the system from a start state – the application launch – to a finish state – application termination. All the possibilities of user interaction are covered, mapping all the possible paths of the system. For the DOCSEDITOR case the Markov chain model used has 53 states representing several combinations between the characteristics of the environment of use (browser and server availability), tree window and possible dialogs with the user.

The next step was to assign the transition rates to the model structure, using the event log information. For this experiment, we considered data from one week of use with 6 users. For each transition in the model, the transition rate was computed as the mean number of occurrences of the transition per execution. The transitions with zero occurrences had the transition rate set to a minimum value, in order to show that these transitions may occur although not recorded in the sample. In this example 135 different transitions among the states were considered. Mostly these transitions consider the type of node selected in the object tree of the application (root, leaf or intermediate node), the existence of a browser installed or active application server, and, of course, the eight different kinds of dialog that may appear during the execution.

Finally, we have the finish and start states representing the launch and termination of DOCSEDITOR execution. An additional transition from the finish to the start state is included to represent the start of a new session⁵. The Markov model is not represented graphically in this paper due to its relative complexity (it will need a graph of 53 nodes and 136 arcs).

4.2.2 Model 2 - Stochastic Automata Network

Differently of the process used in previous model, *i.e.*, specification and events registry analysis, the SAN model was developed by translating the Markov model into a SAN. The first step was to define different automata to each aspect of the software, *i.e.*, the observation of the state classes found in the first model. Thus, we defined five automata (Figure 5):

- the document type automaton, representing the no active document (*No Doc*), titled document active (*Titled*) and untitled document active (*Untitled*);
- the eight possible dialogs windows (from *Dialog 1* to *Dialog 8*) and the absence of dialog window (*ND*);
- the type of node selected in the object tree of the application (*Root*, *Leaf* or intermediate node - *Other*);
- the presence (*Server*) or absence (*No Server*) of active application server;
- the presence (*Browser*) or absence (*No Browser*) of installed browser.

Figure 5 represents graphically the five automata describing the usage model for DOCSEDITOR. For reasons of space we limit our representation of the SAN model in this paper to a partial description, the reader can find the complete SAN model, as well as the Markov chain equivalent model, in the technical report referred in [2]. For clarity, in this figure we do not represent the rates of local events by arc annotations as we did in section 3. There are local transitions rates in every arc drawn in this figure, however we only indicate the synchronized events. It is important to do so, in order to realize the synchronized moves of the automata. The synchronized events *S*, *T*, *U*, *V* and *W* represent changes in the document state confirmed (synchronized) with particular answers from Dialogs 1, 2, 3 and 4.

The synchronized event *R* represents the re-start of the application that implies in the change of the first automaton to the state *No Doc* and a possible change in the browser status (each re-start may be made with or without a browser). It is important to note that several local transitions are functional, so the interaction among automata are not limited to the synchronized events represented.

This model has 324 global states, *i.e.*, the combination of local states of all five automata ($3 \times 9 \times 3 \times 2 \times 2$), but since it is equivalent to the Markov chain model, only 53 of those global states are reachable.

⁵This last included transition also provides the ergodicity of the Markov model.

5. CONCLUSION AND FUTURE WORK

Markov-based usage models have been widely applied to statistical testing, with important benefits, such as described in [10, 12, 13, 16]. Stochastic automata networks add value to traditional Markov chains, both from modeling and computational viewpoints. In this paper, we described an investigation concerning the use of SAN to represent usage models for statistical testing, from the modeling perspective. At this point, some preliminary conclusions are available:

- Some features of the application that were implicit in the Markov chain model were easily made explicit in the SAN-based model - for example, the node type (root, leaf or other). In order to represent such features in the Markov chain, we would need to create many additional states, and the resulting model would be very complex;
- Some automata in the network correspond intuitively to environment requirements, which are very useful for test case generation - for example, the server state; the model captures information about the relative priorities (frequencies) of the different environments that should be tested;
- The conditions associated to the transitions, and their combinations, map directly to test case descriptions.

On the other hand, the high density of information contained in transitions makes model interpretation less intuitive. There are other extensions of traditional Markov-based models for statistical testing; for example, Tian and Lin [12] apply Unified Markov Models (UMM), which add hierarchy to Markov chains, in usage modeling tasks. SAN are not hierarchical, instead, they allow for many different types of relationships among system components, while keeping the description modular. From a computational point of view, SAN, also have higher scalability: for example, in the case of DOCSEDITOR, we can perform model analysis considering many servers and clients in the environment, with little impact in the computational effort to compute model properties. Such flexibility is useful for both statistical and load testing.

The experiment described here is a first case study: the target application is relatively simple, and we focused on the modeling process. The next (already started) steps include: analyzing the computational properties of the model, developing a tool for random test case generation from SAN-based usage models, executing the tests, and evaluating the results. Also, we will develop experiments with more complex target applications.

Acknowledgements

This work is partially supported by research projects of FAPERGS (*PEPS - Avaliação de Desempenho e Confiabilidade de Sistemas Paralelos*), and HP-Brasil/PUCRS (*CPTS - Centro de Pesquisa em Teste de Software*).

6. REFERENCES

- [1] **CPTS Project:** Centro de Pesquisa em Teste de Software (*Research center on software test*). HP-Brasil/PUCRS, <http://www.cpts.pucrs.br>

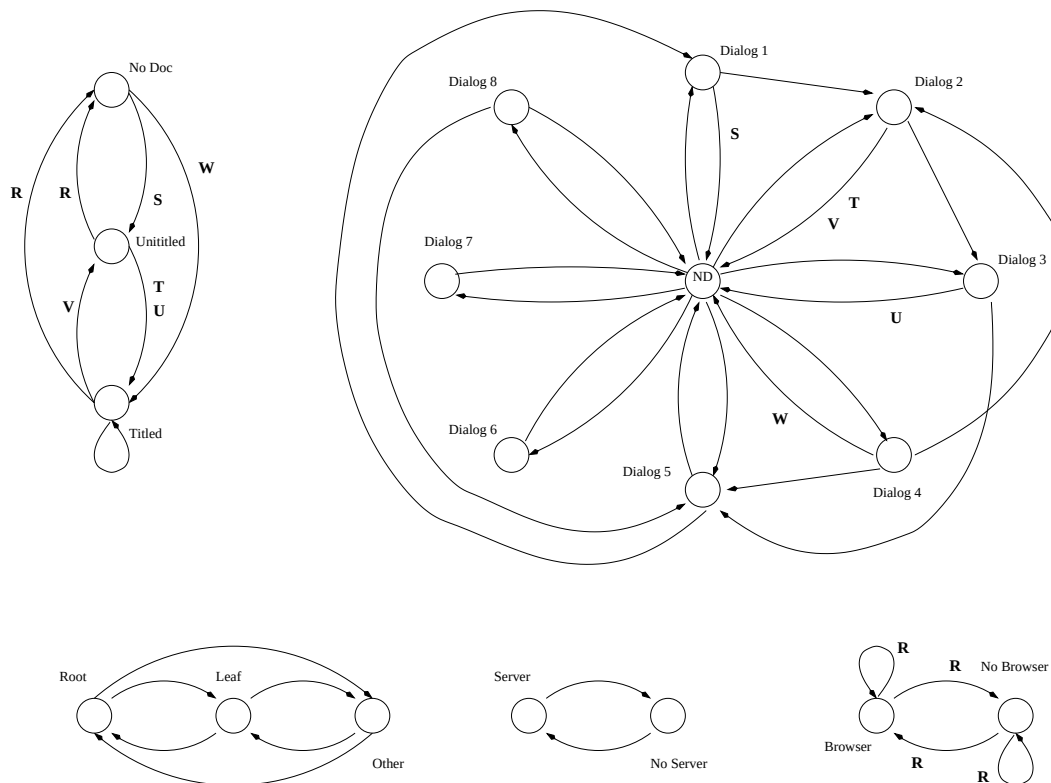


Figure 5: The SAN model for DOCSEDITOR

- [2] A.G.Farina, F.M.Oliveira. **A comparison between usage modeling formalisms for statistical software testing**. Research Report nb. 23, CPPGCC-PUCRS, Porto Alegre, Brazil, March 2002. (available at www.inf.pucrs.br/tr/tr023.pdf)
- [3] P.Fernandes. **Méthodes numériques pour la solution de systèmes Markoviens à grand espace d'états**. INPG, Grenoble, France, 1998. (Ph.D. thesis)
- [4] P.Fernandes, B.Plateau, W.J.Stewart. *Efficient descriptor-vector multiplication in stochastic automata networks*. **Journal of the ACM**, v. 45(3): 381-414, 1998.
- [5] C.Kallepalli, J.Tian. *Measuring and Modeling Usage and Reliability for Statistical Web Testing*. **IEEE Transactions on Software Engineering**, v. 27(11): 1023-1036, 2001.
- [6] J.D.Musa. **Software reliability engineering**. McGraw-Hill, 1998.
- [7] **PEPS Project**: Performance Evaluation of Parallel Systems. ID-INRIA/IMAG/PUCRS, <http://www-apache.imag.fr/software/peps/>
- [8] B.Plateau. **De l'Evaluation du Parallélisme et de la Synchronisation**. Paris-Sud, Orsay, France, 1984. (Ph.D. thesis)
- [9] B.Plateau, K.Atif. *Stochastic automata networks for modelling parallel systems*. **IEEE transactions on software engineering**, v. 17(10): 1093-1108, 1991.
- [10] K.Sayre. **Improved techniques for software testing based on Markov chain usage models**. University of Tennessee, Knoxville, USA, 1999. (Ph.D. thesis)
- [11] W.J.Stewart. **Introduction to the numerical solution of Markov chains**. Princeton University Press, 1994.
- [12] J.Tian, E.Lin. *Unified Markov Models for Software Testing, Performance Evaluation, and Reliability Analysis*. In: **Proceedings of the 4th ISSAT - International Conference on Reliability and Quality in Design**, Seattle, USA, August 12-14, 1998.
- [13] C.Trammel. *Quantifying the reliability of software: statistical testing based on a usage model*. In: **Proceedings of the 2nd ISSES - International Symposium on Software Engineering Standards**, Montreal, Canada, August 21-25, 1995.
- [14] K.S.Trivedi. **Probability & statistics with reliability, queueing and computer science applications**. Prentice-Hall, 1982.
- [15] G.H.Walton, J.H.Poore. *Generating transposition probabilities to support model-based software testing*. **Software Practice and Experience**, v. 30: 1095-1106, John Wiley & Sons, 2000.
- [16] J.A.Whittaker, M.G.Thomason. *A Markov chain model for statistical software testing*. **IEEE Transactions on Software Engineering**, v. 20(10): 812-824, 1994.